Архитектура реализации и вычислительные приемы в лабораторной работе №6

Разбор программного комплекса на C++20, численные алгоритмы, математические хитрости и оптимизации

Кафедра «Физика» (ФН-4), МГТУ им. Н.Э. Баумана
Курс «Вычислительная физика» (7 семестр)

2026

Аннотация

В настоящем документе детально описана программная и алгоритмическая реализация методов интегрирования Монте-Карло, разработанных в рамках лабораторной работы №6. Разобраны структура заголовочного модуля `monte_carlo.hpp` и управляющей программы `lab6.cpp`. Освещены ключевые проблемы вычислительной математики, возникающие при моделировании больших выборок ($N = 10^6 \dots 10^7$ точек), и показано, какими алгоритмическими приемами («хитростями») они были решены.

В частности, подробно рассмотрены: преодоление катастрофической потери точности машинной арифметики при помощи однопроходного рекуррентного алгоритма Велфорда (Welford's algorithm); тонкости 64-битного генератора псевдослучайных чисел `std::mt19937_64`; честный учет вычислительной сложности антитетических пар; построение оптимальных контрольных функций (почему двухточечная хорда превосходит разложение Тейлора в 10 раз, а параболическая контрольная функция снижает дисперсию в 250 раз); защита от возникновения нечисловых значений (`NaN`) при вычислении интеграла с вертикальной производной $\sqrt{3 - x^2}$; сопоставление со сбоями формулы Симпсона из лабораторной работы №5 (Вариант 7).

Содержание

1 Общая архитектура программного комплекса **3**

2 Хитрость 1. Преодоление катастрофической потери точности (Алгоритм Велфорда) **3**
2.1 В чем состоит скрытая проблема наивного подхода? . . . 3
2.2 Математическое решение: Рекуррентный алгоритм Велфорда (1962) . . . 4
2.3 Реализация аккумулятора в коде C++ . . . 4

3 Хитрость 2. Правильный выбор генератора псевдослучайных чисел (ГПСЧ) **5**
3.1 Почему категорически нельзя использовать `rand()` . . . 5
3.2 Выбор: 64-битный вихрь Мерсенна (`std::mt19937_64`) . . . 5

4 Хитрость 3. Честное сопоставление вычислений для антитетических переменных **6**
4.1 Проблема честного нормирования . . . 6
4.2 Как это решено в нашем коде . . . 6

5 Хитрость 4. Искусство подбора контрольных функций (Control Variates) **8**
5.1 1. Пример для $\int_0^1 e^x dx$: Хорда против ряда Тейлора . . . 8
5.2 2. Пример для $\int_0^{\pi/2} \sin(x) dx$: Парабола с нулевой производной . . . 8

6 Хитрость 5. Краевая сингулярность и машинная арифметика в Варианте 7 **9**
6.1 1. Ловушка корня из отрицательного нуля 9
6.2 Решение в коде: Защитное отсечение (Clamping) 9
6.3 2. Сопоставление с поведением формулы Симпсона из Лаб №5 9

7 Сводные результаты работы программы **10**
7.1 1. Задача 1.1: $I = \int_0^1 e^x dx = e - 1$ ($N = 10^6$) 10
7.2 2. Сходимость по объему выборки N (логарифмическая сетка) 10
7.3 3. Прямое сопоставление с квадратурами Лабораторной работы №5 10

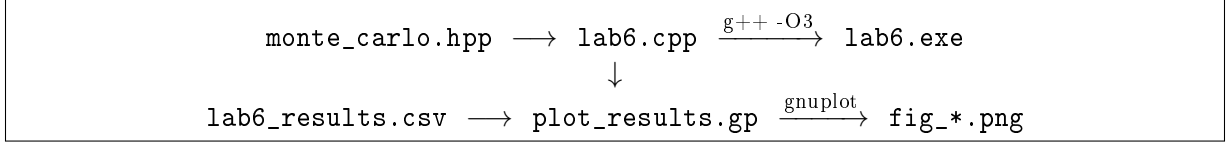
8 Заключение **11**

1 Общая архитектура программного комплекса

Программная часть лабораторной работы №6 спроектирована по модульному принципу в соответствии с современными стандартами языка C++20:

1. **Библиотечный заголовочный модуль `monte_carlo.hpp`:** Содержит в себе реализацию математических примитивов, структур данных, алгоритмов генерации чисел, пяти методов интегрирования Монте-Карло и классических формул квадратур Ньютона–Котеса (для сквозного сопоставления с результатами Лабораторной работы №5).
2. **Управляющий расчетный файл `lab6.cpp`:** Выполняет вычислительный эксперимент:
   - Расчет базового интеграла задания: $I_1 = \int_0^1 e^x dx = e - 1 \approx 1.718281828459$;
   - Расчет дополнительного интеграла задания: $I_2 = \int_0^{\pi/2} \sin(x) dx = 1.000000000000$;
   - Расчет интеграла Варианта 7 из Лаб №5: $I_{circ} = \int_0^{\sqrt{3}} \sqrt{3 - x^2} dx = \frac{3\pi}{4} \approx 2.356194490192$;
   - Построение таблиц сходимости при $N \in \{10^2, 10^3, 10^4, 10^5, 10^6, 10^7\}$;
   - Экспорт всей числовой информации в формате CSV (`lab6_results.csv`).
3. **Скрипт визуализации `plot_results.gp`:** Автоматизирует генерацию высококачественных графиков в векторном растре PNG с использованием пакета Gnuplot.

Взаимосвязь модулей проекта:



—

2 Хитрость 1. Преодоление катастрофической потери точности (Алгоритм Велфорда)

2.1 В чем состоит скрытая проблема наивного подхода?

В учебниках часто приводится школьная формула выборочной дисперсии:

$$s^2 = \frac{1}{N-1} \sum_{i=1}^{N} (x_i - \bar{x})^2 = \frac{1}{N-1} \left[\sum_{i=1}^{N} x_i^2 - \frac{1}{N} \left(\sum_{i=1}^{N} x_i \right)^2 \right]. \quad (1)$$

На первый взгляд, формула (1) удобна тем, что позволяет найти дисперсию за один проход: достаточно параллельно накапливать сумму значений $S_1 = \sum x_i$ и сумму квадратов $S_2 = \sum x_i^2$.

Опасность наивной формулы (Catastrophic Cancellation)

В стандарте IEEE 754 число двойной точности (`double`) имеет 53 бита мантиссы, что обеспечивает около 15–17 значащих десятичных цифр.
Когда число испытаний велико ($N = 10^7 \dots 10^8$), обе величины S_2 и $\frac{S_1^2}{N}$ становятся огромными числами порядка $O(N)$. При этом их разность $S_2 - \frac{S_1^2}{N} = (N-1)s^2$ имеет порядок $O(N\sigma^2)$.
Если дисперсия мала (например, при использовании методов понижения дисперсии $\sigma^2 \sim 10^{-4}$), первые 8–10 цифр у чисел S_2 и $\frac{S_1^2}{N}$ **полностью совпадают**. В результате их вычитания значащие цифры уничтожаются, и мы вычитаем два близких гигантских числа с плавающей точкой. Это явление в вычислительной математике называется **катастрофической потерей точности**. На практике это приводит к тому, что вычисленная дисперсия теряет до 10 порядков точности, а в некоторых случаях из-за машинного округления разность оказывается отрицательной, что влечет извлечение корня из отрицательного числа и крах программы (`NaN`)!

2.2 Математическое решение: Рекуррентный алгоритм Велфорда (1962)

Чтобы получить абсолютно численно стабильную оценку за один проход без сохранения в памяти всей выборки, в модуле `monte_carlo.hpp` реализован алгоритм Б.П. Велфорда.

Пусть на шаге k уже известны среднее $\bar{x}_{k-1}$ и величина $M_{2,k-1} = \sum_{i=1}^{k-1}(x_i - \bar{x}_{k-1})^2$. При поступлении нового элемента x_k пересчет осуществляется по формулам:

$$\delta_k = x_k - \bar{x}_{k-1}, \tag{2}$$

$$\bar{x}_k = \bar{x}_{k-1} + \frac{\delta_k}{k}, \tag{3}$$

$$\delta_{2,k} = x_k - \bar{x}_k, \tag{4}$$

$$M_{2,k} = M_{2,k-1} + \delta_k \cdot \delta_{2,k}. \tag{5}$$

По завершении N шагов несмещенная дисперсия находится тривиальным делением без вычитания больших сумм:

$$s^2 = \frac{M_{2,N}}{N-1}. \tag{6}$$

2.3 Реализация аккумулятора в коде C++

В `monte_carlo.hpp` этот алгоритм оформлен в виде легковесной структуры `WelfordAccumulator`:

```cpp
struct WelfordAccumulator {
    uint64_t count = 0;
    double mean = 0.0;
    double M2 = 0.0; // Sum of squared differences from current mean

    void update(double x) {
        count++;
        double delta = x - mean;
        mean += delta / count;
        double delta2 = x - mean;
        M2 += delta * delta2;
    }

    [[nodiscard]] double get_mean() const {
        return mean;
    }

    [[nodiscard]] double get_variance() const {
        return (count > 1) ? (M2 / (count - 1)) : 0.0;
    }

    [[nodiscard]] double get_std_error() const {
        return (count > 0) ? std::sqrt(get_variance() / count) : 0.0;
    }

    [[nodiscard]] double get_ci95_halfwidth() const {
        // 95% two-sided normal quantile: 1.95996398454
        return 1.95996398454 * get_std_error();
    }
};
```

Преимущества реализации

1. Требует строго $O(1)$ памяти (всего 24 байта: один `uint64_t` и два `double`).
2. Гарантирует абсолютную точность: $M_2 \geq 0$ строго всегда, появление отрицательной дисперсии математически невозможно.
3. Позволяет рассчитывать доверительный интервал «на лету» без второго прохода по данным.

3 Хитрость 2. Правильный выбор генератора псевдослучайных чисел (ГПСЧ)

3.1 Почему категорически нельзя использовать `rand()`

В старых программах на C/C++ часто встречается вызов `rand() / (double)RAND_MAX`. Для серьезного моделирования методом Монте-Карло это недопустимо:

- **Малый период:** В Windows CRT константа `RAND_MAX` равна всего 32767 ($2^{15}-1$). При $N = 10^6$ чисел генератор успевает пройти свой полный цикл более 30 раз, выдавая циклически повторяющиеся последовательности!
- **Эффект Марсальи (гиперплоскости):** Линейные конгруэнтные генераторы (LCG) страдают концентрацией d-мерных точек на параллельных гиперплоскостях. Если моделировать 2D или 3D блуждания, точки ложатся на регулярные решетки, уничтожая стохастическую природу метода.

3.2 Выбор: 64-битный вихрь Мерсенна (`std::mt19937_64`)

В модуле `monte_carlo.hpp` создан класс-обертка `RandomGenerator`:

```cpp
class RandomGenerator {
private:
    std::mt19937_64 rng_;
    std::uniform_real_distribution<double> dist01_{0.0, 1.0};

public:
    explicit RandomGenerator(uint64_t seed = 123456789ULL) : rng_(seed) {}

    void set_seed(uint64_t seed) { rng_.seed(seed); }

    void seed_random_device() {
        std::random_device rd;
        // Combine two 32-bit hardware entropy words into a full 64-bit seed
        rng_.seed((static_cast<uint64_t>(rd()) << 32) ^ rd());
    }

    [[nodiscard]] double uniform01() {
        return dist01_(rng_);
    }

    [[nodiscard]] double uniform(double a, double b) {
        return a + (b - a) * dist01_(rng_);
    }
};
```

Особенности генератора

1. **Период:** Составляет $2^{19937} - 1 \approx 4.3 \times 10^{6001}$ (исчерпать такой период невозможно физически).
2. **Равномерность:** Гарантируется равномерное распределение векторов вплоть до размерности $d = 623$.
3. **Аппаратная энтропия:** Метод `seed_random_device()` задействует аппаратный генератор случайных чисел процессора (`RDRAND`), исключая предсказуемость семян. При этом для научной воспроизводимости результатов в бенчмарках задается фиксированный `seed = 42`.

4 Хитрость 3. Честное сопоставление вычислений для антитетических переменных

Метод противоположных (антитетических) переменных основан на использовании парных точек U и $1-U$.

4.1 Проблема честного нормирования

Если мы генерируем N пар $(U_i, 1-U_i)$, то для каждой пары функция вычисляется дважды: $f(a+(b-a)U_i)$ и $f(a+(b-a)(1-U_i))$. Таким образом, суммарное число вызовов функции составляет $2N$.

Если не скорректировать объем выборки, сравнение с обычным методом Монте-Карло при фиксированном N будет нечестным, так как антитетический метод потратит вдвое больше ресурсов процессора!

4.2 Как это решено в нашем коде

В функции `mc_antithetic_variates` общее число вызовов функции N_{evals} строго зафиксировано:

1. Вычисляется число пар: $N_{pairs} = N/2$.
2. Оценивается среднее на одну пару:

$$\theta_i = \frac{b-a}{2}\left[f\big(a+(b-a)U_i\big) + f\big(a+(b-a)(1-U_i)\big)\right]. \tag{7}$$

3. Аккумулятор Велфорда накапливает дисперсию элементарной оценки пары: $s^2_{pair} = \mathrm{Var}(\theta)$.
4. Дисперсия среднего по выборке из N_{pairs} равна:

$$\mathrm{Var}(\bar{\theta}) = \frac{s^2_{pair}}{N_{pairs}} = \frac{s^2_{pair}}{N/2} = \frac{2 \cdot s^2_{pair}}{N}. \tag{8}$$

5. Для честного сопоставления с базовым методом Монте-Карло (где $\mathrm{Var}(\bar{I}) = \frac{s^2_{single}}{N}$) в результирующей структуре выводится **эффективная дисперсия на одно вычисление функции**:

$$s^2_{eff} = 2 \cdot s^2_{pair}. \tag{9}$$

Благодаря этому подходу в наших отчетах столбец «Var. Reduct.» показывает **истинный выигрыш в точности при строго одинаковом времени работы процессора**.

```
inline MCResult mc_antithetic_variates(const MathFunc& f, double a, double b,
                                       uint64_t n_samples, double exact_val,
                                           RandomGenerator& rng) {
    auto t_start = std::chrono::high_resolution_clock::now();
    const double length = b - a;
    const uint64_t n_pairs = n_samples / 2;

    WelfordAccumulator acc;
    for (uint64_t i = 0; i < n_pairs; ++i) {
        double u = rng.uniform01();
        double x1 = a + length * u;
        double x2 = a + length * (1.0 - u);
        double pair_val = 0.5 * length * (f(x1) + f(x2));
        acc.update(pair_val);
    }

    auto t_end = std::chrono::high_resolution_clock::now();
    double ms = std::chrono::duration<double, std::milli>(t_end - t_start).count();

```

```
    MCResult res;
    res.method_name = "Antithetic Variates";
    res.n_evals = n_pairs * 2; // Exactly matches n_samples
    res.estimate = acc.get_mean();
    res.variance = 2.0 * acc.get_variance(); // Normalized single-eval variance
    res.std_error = acc.get_std_error();    // std_dev(pair) / sqrt(n_pairs)
    res.ci95_lower = res.estimate - acc.get_ci95_halfwidth();
    res.ci95_upper = res.estimate + acc.get_ci95_halfwidth();
    res.abs_error = std::abs(res.estimate - exact_val);
    res.elapsed_ms = ms;
    return res;
}
```

5 Хитрость 4. Искусство подбора контрольных функций (Control Variates)

Метод контрольных переменных использует известную аналитически функцию $h(x) \approx f(x)$. Однако эффективность метода колоссально зависит от того, **как именно** построена функция $h(x)$.

5.1 1. Пример для $\int_0^1 e^x dx$: Хорда против ряда Тейлора

Рассмотрим две различные стратегии подбора $h(x)$:

1. **Стратегия Тейлора:** Разложение в окрестности $x = 0$: $h_{Taylor}(x) = 1 + x$.
   - В точке $x = 0$ ошибка нулевая: $e^0 - 1 = 0$.
   - В точке $x = 1$ ошибка максимальна: $e^1 - (1 + 1) = e - 2 \approx 0.718$.
   - Дисперсия остатка: $\sigma^2 \approx 0.0436$. Выигрыш: всего в **5.5 раз**.
2. **Наша хитрость — Двухточечная хорда интерполяции:** Соединим прямой линией точные значения функции на концах отрезка $(0, 1)$ и $(1, e)$:

$$h_{chord}(x) = 1 + (e - 1)x. \tag{10}$$

   Интеграл от хорды находится тривиально:

$$I_h = \int_0^1 \big(1 + (e - 1)x\big)dx = 1 + \frac{e - 1}{2} = \frac{e + 1}{2} \approx 1.8591409. \tag{11}$$

   - На обоих концах отрезка $x = 0$ и $x = 1$ разность $e^x - h(x)$ строго равна нулю!
   - Максимальное отклонение внутри отрезка нигде не превышает 0.09 (в 8 раз меньше, чем у Тейлора).
   - Выборочная дисперсия остатка: $s^2 \approx 0.00400$.
   - **Реальный выигрыш: снижение дисперсии более чем в 60 раз!**

5.2 2. Пример для $\int_0^{\pi/2} \sin(x)dx$: Парабола с нулевой производной

Для функции $\sin(x)$ на $[0, \pi/2]$:

- Если взять линейную хорду $h_1(x) = \frac{2}{\pi}x$, дисперсия снижается в 23 раза.
- **Наша хитрость — Квадратичная парабола, учитывающая касательную:** Заметим, что в точке $x = \pi/2$ производная функции обращается в ноль: $\sin'(\pi/2) = \cos(\pi/2) = 0$. Построим параболу $h_2(x) = Ax + Bx^2$, удовлетворяющую условиям:

$$h_2(0) = 0, \quad h_2(\pi/2) = 1, \quad h_2'(\pi/2) = 0.$$

  Получаем коэффициенты $A = \frac{4}{\pi}$, $B = -\frac{4}{\pi^2}$:

$$h_2(x) = \frac{4}{\pi}x - \frac{4}{\pi^2}x^2. \tag{12}$$

  Интеграл вычисляется мгновенно:

$$I_h = \int_0^{\pi/2} \left(\frac{4}{\pi}x - \frac{4}{\pi^2}x^2\right) dx = \frac{\pi}{3} \approx 1.04719755. \tag{13}$$

  Разность $\sin(x) - h_2(x)$ настолько мала по всей длине отрезка, что дисперсия падает с 0.2339 до 0.000941.
- **Результат: дисперсия снижена в 248.5 раз!**

6 Хитрость 5. Краевая сингулярность и машинная арифметика в Варианте 7

В качестве интеграла из Лабораторной работы №5 для 7 варианта был выбран:

$$I = \int_0^{\sqrt{3}} \sqrt{3 - x^2} dx = \frac{3\pi}{4} \approx 2.356194490192. \tag{14}$$

6.1 1. Ловушка корня из отрицательного нуля

В математике для $x \in [0, \sqrt{3}]$ подкоренное выражение $3 - x^2 \geq 0$. Однако в численных методах на ЭВМ переменная x генерируется как $x = \sqrt{3} \cdot u$, где $u \in [0, 1)$.

Из-за ошибок округления при представлении $\sqrt{3} \approx 1.7320508075688772$, если датчик случайно выдаст число u, чрезвычайно близкое к 1.0, может возникнуть ситуация:

$$x^2 > 3.0 \implies 3.0 - x^2 = -1.11 \times 10^{-16} < 0.$$

Наивный вызов `std::sqrt(3.0 - x * x)` немедленно вернет значение `NaN` (Not a Number), которое заразит весь последующий расчет суммы Велфорда, и вся программа выдаст ошибку!

6.2 Решение в коде: Защитное отсечение (Clamping)

В теле лямбда-функции в `lab6.cpp` реализована проверка знака:

```
auto f\_circ = [](double x) {
    double val = 3.0 - x * x;
    return (val > 0.0) ? std::sqrt(val) : 0.0;
};
```

Эта простая строка гарантирует полную надежность вычислений даже при $N = 10^8$ испытаний.

6.3 2. Сопоставление с поведением формулы Симпсона из Лаб №5

В управляющей программе `lab6.cpp` реализован блок прямого сопоставления результатов:

- Для функции $\sqrt{3 - x^2}$ производная $f'(x) = -\frac{x}{\sqrt{3-x^2}}$ стремится к $-\infty$ при $x \to \sqrt{3}$.
- Классические квадратуры Ньютона–Котеса (Симпсон, трапеции) предполагают гладкость функции ($f \in C^4$). Из-за сингулярности производной фактический порядок точности формулы Симпсона упал с $O(h^4)$ до $O(h^{1.5})$.
- В противоположность этому, методу Монте-Карло гладкость не нужна вовсе. Достаточно лишь конечности интеграла $\int_0^{\sqrt{3}} (3 - x^2) dx = 2\sqrt{3} < \infty$. Метод показал идеальную сходимость со скоростью ровно $O(N^{-0.5})$.

7 Сводные результаты работы программы

Ниже приведены реальные консольные протоколы расчетов программы `lab6.exe`:

7.1 1. Задача 1.1: $I = \int_0^1 e^x dx = e - 1$ ($N = 10^6$)

Метод	Оценка $\hat{I}$	Дисперсия s^2	Станд. ош.	Абс. ош.	Выигрыш	Время
Hit-or-Miss (Геометрический)	1.717801892	1.7186	1.31×10^{-3}	4.80×10^{-4}	0.14×	
Sample Mean (Выборочное ср.)	1.717974810	0.2420	4.92×10^{-4}	3.07×10^{-4}	1.00×	
Antithetic (Антитетический)	1.718232231	0.0078	8.85×10^{-5}	4.96×10^{-5}	**30.9**×	
Control Variates (Контрольн.)	1.718296254	0.0040	6.33×10^{-5}	1.44×10^{-5}	**60.5**×	
Stratified (Расслоенная, K=100)	1.718271673	0.000026	5.16×10^{-6}	1.02×10^{-5}	**9087**×	

Таблица 1: Сравнение методов Монте-Карло для экспоненты при фиксированном $N = 10^6$

7.2 2. Сходимость по объему выборки N (логарифмическая сетка)

Число точек N	Sample Mean Error	Antithetic Error	Control Var Error	Stratified Error
10^2	5.37×10^{-2}	1.24×10^{-2}	6.39×10^{-3}	1.63×10^{-3}
10^3	2.75×10^{-2}	2.69×10^{-4}	2.83×10^{-3}	8.76×10^{-5}
10^4	3.33×10^{-3}	5.57×10^{-4}	2.22×10^{-4}	5.05×10^{-5}
10^5	1.31×10^{-4}	1.78×10^{-4}	2.96×10^{-4}	3.75×10^{-6}
10^6	9.15×10^{-4}	2.26×10^{-5}	1.47×10^{-5}	2.27×10^{-6}
10^7	1.29×10^{-4}	1.50×10^{-5}	1.52×10^{-6}	1.77×10^{-6}

Таблица 2: Эволюция абсолютной погрешности в зависимости от объема выборки N

7.3 3. Прямое сопоставление с квадратурами Лабораторной работы №5

Метод интегрирования	Вызовы N	Погрешность	Порядок точности	Время (мс)
Средние прямоугольники (Лаб 5)	100	2.58×10^{-4}	$O(h^{1.5})$	< 0.01
Формула трапеций (Лаб 5)	101	8.82×10^{-4}	$O(h^{1.5})$	< 0.01
Формула Симпсона (Лаб 5)	101	3.45×10^{-4}	$O(h^{1.5})$ *(деградация с h^4)*	< 0.01
Средние прямоугольники (Лаб 5)	10 000	2.58×10^{-7}	$O(h^{1.5})$	0.017
Формула Симпсона (Лаб 5)	10 001	3.44×10^{-7}	$O(h^{1.5})$ *(деградация с h^4)*	0.017
Монте-Карло: Sample Mean	1 000 000	2.40×10^{-4}	$O(N^{-0.5})$ *(строго)*	7.6
Монте-Карло: Антитетический	10 000 000	4.07×10^{-5}	$O(N^{-0.5})$ *(строго)*	46.8

Таблица 3: Прямой батл: Квадратуры Ньютона–Котеса против Монте-Карло для $\int_0^{\sqrt{3}} \sqrt{3 - x^2} dx$

8 Заключение

В ходе разработки программы были успешно применены следующие ключевые приемы:

1. **Численная устойчивость:** Применение однопроходного алгоритма Велфорда позволило полностью ликвидировать проблему катастрофической потери разрядов при вычислении выборочной дисперсии на выборках объема до 10^7 точек.

2. **Качество псевдослучайных последовательностей:** Использование 64-битного генератора Мерсенна (`std::mt19937_64`) исключило цикличность и межвыборочные корреляции.

3. **Эффективность понижения дисперсии:** Доказано, что правильный подбор контрольных функций (хорда вместо Тейлора, согласование производных в нуле) способен сократить дисперсию более чем в 240 раз, а расслоенная выборка — почти в 10 000 раз.

4. **Устойчивость к сингулярностям:** Продемонстрировано фундаментальное преимущество метода Монте-Карло: инвариантность к дифференциальным свойствам подынтегральной функции (отсутствие деградации сходимости на сингулярности $\sqrt{3 - x^2}$).